第1章 什么是 AI

> 本章目标：把"AI"这个模糊的大词拆成八级台阶，让你以后写代码、调接口、向别人解释方案时，能准确说出自己站在哪一层、这一层是"人写死的"还是"从数据学出来的"。

1.1 一把梯子：从写死的规则到学出来的能力

初学者最容易犯的错，是把"AI"当成一个东西。实际上它是一串层级，每一层和上一层的差别只有一个：**哪些东西由人固定下来，哪些东西由数据决定。**

从下往上走，人写的部分在缩小，数据学到的部分在扩大：

层级	人固定什么	数据学到什么	典型例子
普通程序	全部逻辑与分支	什么都不学	计算器、排序、PID 控制、状态机
经典 AI（搜索与规则）	搜索空间、规则表、打分函数	基本不学，或只调少量权重	A* 路径规划、Alpha-Beta 博弈搜索、专家系统
机器学习	用什么特征、学什么目标	一组参数	垃圾邮件过滤、SVM 分类、房价回归
深度学习	网络结构（层数、连接方式）	从原始数据到特征的整个映射	CNN 图像分类、语音识别
大模型	架构 + 训练目标（如"预测下一个词"）	几乎全部能力，且通用	GPT、Claude、Qwen、DeepSeek
聊天产品	界面、系统提示、拒答与安全策略	不再学习，只消费模型	各家网页版与 App
Agent	工具清单、循环规则、终止条件	决策过程来自模型	Claude Code、Codex CLI、Gemini CLI
Harness	上下文管理、权限、日志、工具协议	不学	把模型装进去运行的外壳

> 要点：越往上，"人写的"并没有消失，只是换了位置——从写规则，变成写架构、写目标函数、写工具边界。判断一个系统在梯子的哪一层，看的是"人还在哪里干预"。

注意一个反直觉的事实：**上面三层又把大量人写的东西加了回来。** 大模型的"学习"发生在训练阶段，而聊天产品、Agent、Harness 都是训练之后人为搭出来的。所以"大模型很聪明"和"你的 Agent 好用"是两件独立的事——后者往往取决于 Harness 写得怎么样，这也是本课后面几章的重点。

1.2 传统算法 vs 机器学习：同一任务，两种做法

判断该用哪一种，问一个问题：**这条规则，你能不能用几句话写清楚？**

任务	传统算法怎么做	机器学习怎么做	实践中怎么选
路径规划	Dijkstra / A* 在图或栅格上搜最短路	学"走这一格的代价"，或端到端学策略	仍以经典搜索为主，ML 用来改代价函数
垃圾邮件过滤	关键词黑名单 + 人工打分	用大量标注邮件学权重	ML，因为规则永远追不上新花样
推荐	手写规则：热销榜、同作者	学用户向量与物品向量的匹配	ML，规则只做冷启动兜底
图像分类	人工设计特征（边缘、纹理）+ 分类器	端到端从像素学到类别	深度学习，人工特征已全面落后
游戏 NPC	状态机、行为树、脚本	强化学习策略，或 LLM 驱动对话	两种都用：行为用状态机，对话用模型
音频降噪	维纳滤波、谱减法等信号处理	学"带噪 → 干净"的映射	传统方法可解释、延迟低；学习法效果上限更高

规律很清晰：**规则明确、可枚举、要可解释的，用传统算法；规则复杂到写不出来、但有大量样本的，用机器学习。** 注意最后一行——传统方法不是"过时的东西"，在延迟、功耗、可解释性敏感的场合它经常更合适。机器人控制里的运动学求解就是典型例子。

1.3 五个最容易混的词

它们经常被混着说，但完全是五种东西：

- **模型（model）**：一堆数字（参数）。文件形态可能是几个 GB 的权重文件。它只会做一件事：给定输入，算出一个输出。它不听话、不稳定、也不会主动做事。
- **聊天产品（product）**：网页或 App。模型只是它的一个零件，外面还有界面、系统提示、历史记录、审核策略。你在产品里看到的行为，很多不是模型本身决定的。
- **API**：让程序调用模型的接口。你付钱、发请求、拿回应。它把"模型"变成"可以被代码使用的东西"。
- **Agent**：让模型在循环里调用工具的程序。模型决定下一步做什么，程序负责执行并回灌结果。它比聊天多了一个"手脚"。
- **普通程序**：不包含模型的代码。排序、读文件、开电机的程序都算。

一个常见的误解是"我接入了 API 就等于有了 Agent"。不是。API 是单次问答；Agent 是循环加工具。中间那层脚手架，正是后面章节要讲的东西。

1.4 模型类型速查

"AI"在工程上其实是一族专门化工具的统称。看到一个模型，先问：**输入是什么，输出是什么。**

模型类型	输入 → 输出	典型用途
LLM 大语言模型	文本 → 文本	对话、写代码、摘要、抽取、推理
ASR 语音识别	音频 → 文本	语音输入、字幕、语音指令
TTS 语音合成	文本 → 音频	朗读、语音助手发声
生图	文本（可带参考图）→ 图像	配图、概念设计、贴图素材
文生视频	文本或图 → 视频	短片素材、分镜预演
目标检测	图像 → 若干边界框 + 类别	机器人抓取、安防、工业质检
语义分割	图像 → 逐像素类别	可抓取区域、医学影像
深度估计	单张图像 → 深度图	单目避障、三维重建
分类	图像或文本 → 类别 + 概率	缺陷检测、情感分析
Embedding 嵌入	文本或图像 → 一串数字（向量）	检索、聚类、去重、推荐
音源分离	混合音频 → 多条音轨	人声与伴奏分离、降噪前处理
降噪	带噪音频 → 干净音频	通话、录音清洗

把这个表背下来意义不大，**养成"先问输入输出"的习惯**才是关键。工程上九成的选型错误，都源于没看清任务到底属于哪一格。

1.5 大模型与小模型

大模型和小模型不是"好和坏"，是**三种约束下的取舍**：能力、成本、部署位置。

- **大模型**：参数多，泛化强，一个模型能干很多种任务；代价是推理慢、吃显存、通常只能放在云端。
- **小模型**：专精一件事，跑得快、能塞进手机或开发板；代价是只能干那一件事，换任务就要换模型。

机器人上常见的组合是：小模型在本地做实时感知（检测、分割），大模型在云端做规划和对话。**让合适大小的模型干合适的事**，比一味追大模型更专业。

1.6 多模态

"多模态（multimodal）"指的是能同时处理多种输入类型：文本、图像、音频、视频。

这里要分清楚两件事：

- **真正的多模态模型**：在同一个模型里联合训练多种输入，能理解"这张图里第三排左边那个零件是什么"。
- **拼装流水线**：ASR 转文字 → LLM 处理 → TTS 输出。每一段都是单模态模型，串起来"看起来"多模态。

两者的差别在出错时会暴露：流水线里前一段错了，后面全错，而且错误会被伪装成"模型理解错了"。设计系统时要想清楚自己用的是哪一种。

（图片生成背后的几大技术流派——扩散模型、GAN、VAE、自回归——放在第十二章多模态里讲，这里只需知道它们不是同一个东西。）

1.7 训练与推理

这是两个必须分开的阶段。

> 类比：**训练像高考备考，推理像考试答题。** 备考要几个月、要刷几万道题、要反复订正错题；答题只有两小时，写完交卷，脑子里的知识不会再变。

对应到工程上：

	训练（training）	推理（inference）
干什么	用数据反复调整参数	用固定参数算输出
参数变不变	一直在变	不变
花什么	算力、时间、数据、电费	延迟、显存、每次调用的费用
在哪做	集群、机房、离线	云端服务，或本地设备
谁能做	少数机构	所有人都能做

这个区分能解释很多现实问题：为什么"模型知识截至某个时间"——因为备考结束了，之后的新题它没见过；为什么微调（fine-tuning）很贵——那相当于让模型重新补一段课；为什么同一个模型在不同设备上速度差很多——推理的开销取决于硬件，不取决于模型聪不聪明。

1.8 为什么"预测下一个词"看起来像理解

大语言模型的训练目标朴素得让人失望：**给定前面的文字，预测下一个词（token）。** 就是这么一件事，重复亿万次。

但要做到这件事，光靠背句子是不够的。要准确预测下一个词，模型必须隐式地掌握语法结构、事实关联、常见推理步骤的统计规律——因为这些都是"下一个词"出现概率的一部分。于是梯子往上长出了一层谁也没预料到的能力：它开始能回答问题、写代码、做多步推理。

同时必须诚实：**这不等于人类意义上的理解。** 它是在海量文本上压缩出的一个"够用的统计模型"，没有真实的意图，也没有事实核查机制。它的错误往往不是随机噪声，而是系统性偏差——越是自己不熟悉的领域，越会流畅地编造。后面讲幻觉、讲评估、讲 Harness 时，这个性质会反复出现。

课后题

1. （概念）用"人固定什么 / 数据学到什么"的框架，说明"普通程序"和"深度学习"的区别。再说明为什么 Agent 这一层反而不怎么涉及"学习"。
2. （概念）判断下列系统分别在梯子的哪一层，并说明理由：① 用 A* 找最短路径的扫地机；② 用 CNN 识别零件缺陷的质检相机；③ 一个能自己读文件、改代码、跑测试的命令行工具；④ 手机上的语音输入法。
3. （动手）打开一个你常用的 AI 聊天产品，找出三个"不属于模型本身"的行为（比如拒答、格式、引用来源），推测它们分别由哪一层实现。
4. （动手）从 1.4 的表里挑三个模型类型，各找一个真实存在的产品，写出它的输入和输出。
5. （思考）有人说"模型越大越好"。结合 1.5，说明这句话在什么条件下成立、在什么条件下不成立。给出一个你会选择小模型的场景。
6. （思考）如果"预测下一个词"就足以产生看起来像理解的行为，那我们该如何判断一个模型"真的懂了"？试着提出一个你认为靠谱的检验方法，并说明它可能被怎样绕过。